

Project 2: Simple Motor Protection System

*Current-Sensing Overcurrent Protection with Emergency Stop and LED Status Indication
(ESP32 + PlatformIO)*

By Hiba Banat

Table of Contents

1. Abstract.....	3
2. Introduction	3
3. Objectives.....	4
4. System Design Overview.....	4
5. Bill of Materials	6
6. Wiring & Pin-Out	7
7. Methodology.....	8
8. Firmware (VS Code + PlatformIO)	10
9. Testing & Verification (Definition of Done)	13
10. Recommendations & Schematic Notes	14
11. Conclusion.....	15
References	15

Abstract

This project designs and implements a low-cost motor protection system built around an ESP32 microcontroller. The system continuously monitors the operating current drawn by a DC motor and automatically disconnects the motor's power supply through a relay whenever the current exceeds a pre-defined safe threshold, protecting both the motor and the connected load from thermal and mechanical damage. A hardware-wired emergency-stop (E-stop) push button provides an instantaneous, software-independent means of cutting power to the motor at any time. Four LED indicators communicate the system's operating state at a glance — Running, Stopped, High Current (fault), and Low Current (notice) — giving an operator immediate visual feedback without needing a laptop or serial monitor. The system is developed in C++ using the Arduino framework inside VS Code with the PlatformIO extension, and is documented with a full bill of materials, wiring reference, and a testing procedure mapped directly to the project's Definition of Done.

Introduction

Electric motors are among the most common — and most failure-prone — components in both industrial and hobbyist electromechanical systems. Left unprotected, a motor can be damaged or destroyed in seconds by conditions that are entirely preventable with basic monitoring circuitry. This project addresses that gap directly by giving a small motor real overcurrent protection and a manual safety cutoff, the same fundamental protection scheme used in industrial motor starters and motor circuit protectors, scaled down to an educational hardware platform.

Motors need dedicated protection for several closely related reasons:

- **Excess heat generation:** Current flowing through a motor's windings generates I^2R heating. Any sustained current above the motor's rated value raises winding temperature beyond its insulation rating, degrading the windings and eventually causing insulation breakdown or a short circuit.
- **Overload conditions:** A mechanical overload — a jammed shaft, a stalled load, or an over-torqued coupling — forces the motor to draw far more current than normal while barely rotating (or not rotating at all), which is precisely the condition current monitoring is best at catching.
- **Overcurrent / short-circuit events:** A winding short or a wiring fault can cause a sudden current spike large enough to damage the motor, the wiring, and the power supply within milliseconds if nothing intervenes.
- **Equipment protection:** The motor is rarely the only thing at risk — gearboxes, belts, driven machinery, and the power supply itself can all be damaged by a fault that is allowed to persist.
- **Operator and system safety:** A motor that keeps running during a fault, or that cannot be shut down immediately when something goes wrong, is a safety hazard. An easily reachable, hard-wired emergency stop is standard practice on essentially all powered machinery for this reason.

This report documents the complete design, the hardware bill of materials, the firmware, and the verification steps used to satisfy the project's Definition of Done.

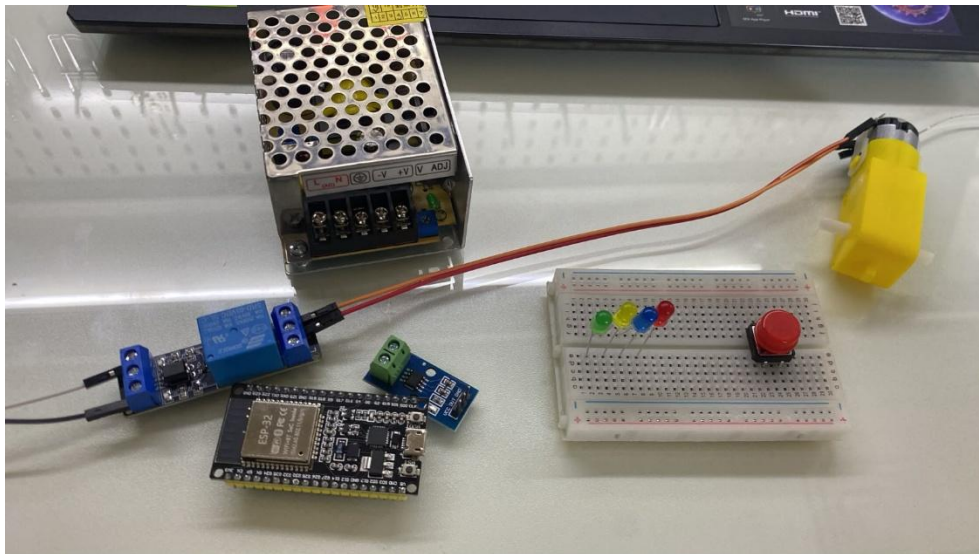
Objectives

- **Design a motor protection system:** built around continuous current monitoring rather than a simple on/off switch.
- **Monitor motor current:** in real time using a dedicated current-sensing IC rather than inferring it indirectly.
- **Detect overcurrent conditions:** against a configurable threshold chosen from the motor's rated current.
- **Disconnect the motor using a relay:** so that the protection action removes power at the source rather than merely stopping a control signal.
- **Implement an emergency stop:** that is wired to cut motor power directly and instantly, independent of the microcontroller's firmware state.
- **Display system status using LEDs:** so the four operating states (Running, Stopped, High Current, Low Current) are visible without a serial monitor.

System Design Overview

The system is organized around a simple, four-state supervisory loop running on the ESP32. A current sensor placed in series with the motor's power line continuously reports the motor's current draw to the microcontroller's analog/I2C input. On every loop iteration, the firmware compares the measured current against two thresholds and against the state of the E-stop button, then drives the relay and the four status LEDs accordingly.

4.1 Functional Block Diagram (described)



[POWER SUPPLY] → [RELAY (NO contact)] → [MOTOR] → back to [POWER SUPPLY] (return line)

[CURRENT SENSOR] is placed in series with the motor's positive supply line, between the relay's NO contact and the motor terminal, so it reads only the motor's current — not the ESP32's own supply current.

[ESP32] reads the current sensor (I2C or analog), reads the E-stop button (digital input, hardware-wired in series with the relay driver supply as a fail-safe), and drives the relay control pin plus the four LED pins.

[E-STOP BUTTON] is wired two ways simultaneously: (1) in series with the relay module's own power feed, so pressing it physically removes power from the relay driver and de-energizes the relay regardless of what the firmware is doing; and (2) into a digital input pin, so the firmware can detect that an E-stop event occurred and latch the Stopped state and the correct LED.

4.2 System States & LED Logic

State	Relay	LED Lit	Trigger Condition	Notes
Stopped	OPEN (de-energized)	Red	System idle at boot, or E-stop pressed, or a fault is latched	Safe default state — motor cannot run unless explicitly cleared
Running	CLOSED (energized)	Green	Measured current is between the Low and High thresholds	Normal, healthy operating condition
High Current (fault)	OPEN — forced by firmware	Yellow/Orange	Measured current > High threshold	Latched: relay stays open until the fault is manually reset
Low Current (notice)	CLOSED — motor keeps running	Blue	Measured current < Low threshold while motor is commanded ON	Not a fault by design — flags a no-load/under-load condition for the operator's attention

Only the High Current condition removes power — this matches the project brief directly: current above the limit trips the relay, while current that stays low simply keeps the motor running (with the Blue LED flagging the condition for visibility). The Low Current notice is useful in practice because it can reveal a slipped belt, a disconnected mechanical load, or a wiring fault long before it becomes a bigger problem.

4.3 Choosing the Overcurrent Threshold

The High Current trip point should sit comfortably above the motor's normal running current but below its stall/locked-rotor current, so that normal starting transients do not cause nuisance trips while genuine overloads are still caught quickly. A practical rule of thumb — the same one used when sizing thermal-magnetic motor protectors industrially — is to set the trip point at roughly 125–150% of the motor's rated (nameplate) running current.

- **Example:** for a small 12 V DC motor with a rated running current of about 0.8–1.0 A, a High Current threshold of 1.5 A gives a reasonable safety margin above normal operation while still tripping well before the motor's stall current (which for small brushed DC motors is often 4–8x the rated current).
- **Low Current threshold:** should sit just above the sensor's noise floor and below the motor's expected no-load current — a starting value of about 0.10 A works well for a small motor and can be tuned after the first test run.

Both values are implemented as named constants in the firmware (see Section 8) so they can be tuned for a different motor without touching any other logic.

Bill of Materials

Component	Qty	Recommended Part	Purpose
Microcontroller	1	ESP32 Dev Board (WROOM-32)	Runs the monitoring/control firmware
Current sensor	1	INA219 I2C current/power sensor breakout (or ACS712 Hall-effect module as an alternative)	Measures real-time motor current
Relay module	1	5 V single-channel relay module, opto-isolated, 3.3 V-logic-compatible trigger	Switches the motor's power line ON/OFF
DC motor (test load)	1	Small 12 V brushed DC motor	The protected load for the demonstration
Push button (E-stop)	1	Panel-mount momentary push button, Normally Closed (NC) contact	Hardware emergency stop
LEDs	4	5 mm LEDs — Green, Red, Yellow, Blue	Running / Stopped / High Current / Low Current indicators
Resistors (LED)	4	330 Ω , 1/4 W (for a 3.3 V GPIO drive)	Current-limiting for each LED
Resistor (pull-up, if needed)	1	10 k Ω	E-stop digital input pull-up (or use ESP32 internal pull-up)
DC power supply (motor)	1	Regulated 12 V DC bench/wall supply, sized per load	Powers the motor circuit independently of the ESP32
Breadboard & jumper wires	1 set	Standard prototyping breadboard + M-M/M-F jumpers	Prototype assembly
USB cable	1	Micro-USB or USB-C (per dev board)	Programming and powering the ESP32
Flyback diode (if not on relay module)	1	1N4007	Protects the relay driver transistor from coil kick-back

Note on the current sensor choice: the INA219 communicates over I2C and is natively 3.3 V-compatible, which avoids the voltage-divider circuit that an ACS712 module would otherwise need before its analog output can be safely read by the ESP32's 3.3 V ADC input. Either sensor is a valid choice for this project; the firmware in Section 8 is written for the INA219.

Wiring & Pin-Out

6.1 ESP32 GPIO Assignment

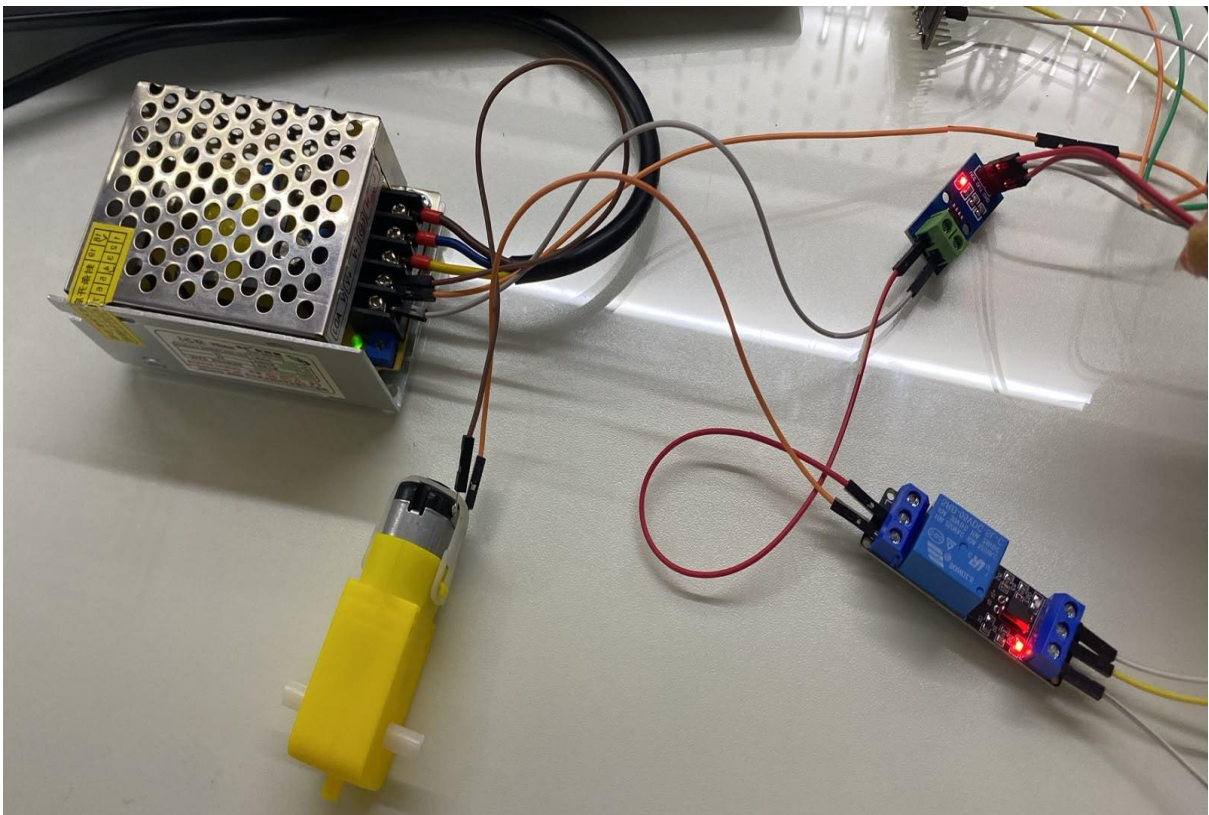
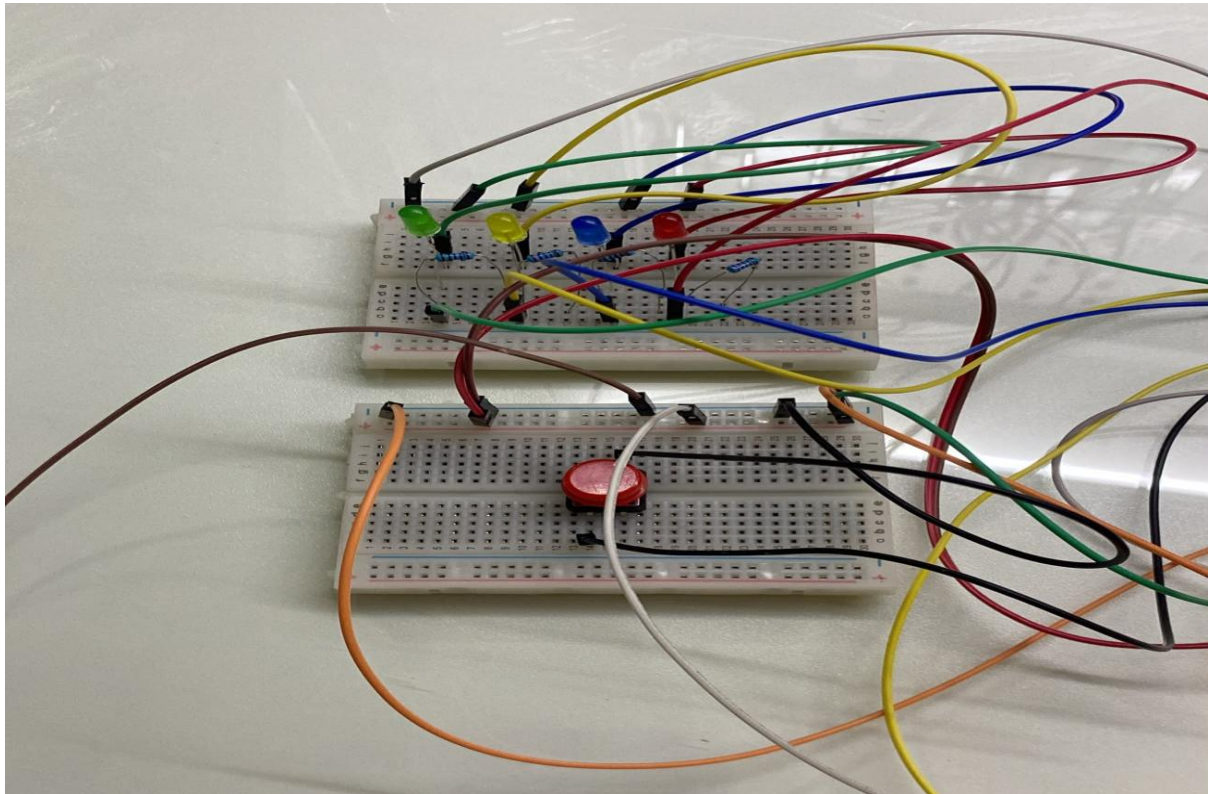
Signal	ESP32 Pin	Notes
Relay control	GPIO 26	Digital output; drives the relay module's IN pin
E-stop input	GPIO 27	Digital input, internal pull-up enabled; wired to the NC button's signal contact
LED — Running (Green)	GPIO 16	Digital output through a 330 Ω resistor
LED — Stopped (Red)	GPIO 19	Digital output through a 330 Ω resistor
LED — High Current (Yellow)	GPIO 17	Digital output through a 330 Ω resistor
LED — Low Current (Blue)	GPIO 18	Digital output through a 330 Ω resistor
Current sensor	GPIO 34	I2C data line to the INA219 breakout

6.2 Power Circuit Wiring (Motor Side)

- 12 V supply (+) → Relay module COM terminal.
- Relay module NO terminal → INA219 current sensor's high-side (Vin+) terminal.
- INA219 low-side (Vin-) terminal → Motor (+) terminal.
- Motor (–) terminal → 12 V supply (–) / common ground.
- ESP32 GND, INA219 GND, relay module GND, and the 12 V supply's (–) rail must all share a single common ground point.

6.3 E-Stop Wiring (Fail-Safe Pattern)

- Run the relay module's own 5 V/3.3 V supply feed through the E-stop button's NC contacts before it reaches the relay module's VCC pin. Pressing the button breaks this feed and physically de-energizes the relay driver, cutting motor power even if the ESP32 has crashed or is mid-reset.
- Separately, wire the same NC button's signal side to GPIO 27 with the ESP32's internal pull-up enabled (INPUT_PULLUP), so the pin reads HIGH normally and LOW the instant the button is pressed.
- This dual wiring means the emergency stop works even if the firmware is not running, while still letting the firmware detect the event and latch the correct LED and state.



Methodology

9. Size and set up the motor's DC power supply, following a load-and-margin calculation (nominal current + inrush + ~25% safety margin) to pick a supply rating that comfortably covers the motor and relay coil.
10. Assemble the hardware on a breadboard: mount the ESP32, relay module, INA219 breakout, LEDs with resistors, and the E-stop button.
11. Connect the current sensor in series with the motor's positive supply line (never in parallel), and connect its I2C lines (SDA/SCL) to the ESP32.
12. Connect the relay so its NO contact switches the motor's positive supply line, and confirm the relay module's trigger voltage matches the ESP32's 3.3 V GPIO output.
13. Connect the E-stop push button using the dual fail-safe pattern described in Section 6.3.
14. Connect the four LEDs, each through its own 330 Ω current-limiting resistor, to the four assigned GPIO pins.
15. Write and upload the firmware from VS Code using the PlatformIO extension, targeting the esp32dev board definition.
16. Test normal operation: power the motor through the relay with no additional load and confirm the Green (Running) LED lights and the motor turns.
17. Simulate an overcurrent condition by briefly and carefully loading or slowing the motor shaft (e.g., lightly gripping it by hand for a moment, or adding a small mechanical load) to push current above the High threshold, and confirm the relay opens and the Yellow (High Current) LED latches on.
18. Verify the emergency stop by pressing the E-stop button during normal running and confirming the motor stops immediately and the Red (Stopped) LED lights, then confirm the motor does not restart until the fault/E-stop condition is cleared and the system is reset.
19. Verify the Low Current notice by running the motor completely unloaded (or disconnecting its mechanical load) and confirming the Blue LED lights while the motor keeps running, per the project's Low Current behavior.
20. Clean up the code (remove debug prints, add comments, confirm named constants are used for every threshold and pin) and produce the final wiring schematic in Fritzing, EasyEDA, or KiCad for submission.

Firmware (VS Code + PlatformIO)

The firmware is organized as a standard PlatformIO project. Create the project in VS Code (PlatformIO: New Project → board: "Espressif ESP32 Dev Module" → framework: Arduino) and replace the generated files as shown below.

8.1 Project Structure

```
motor-protection-system/
├── platformio.ini
├── src/
│   └── main.cpp
└── include/
    └── (unused)
```

8.2 platformio.ini

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
```

8.3 src/main.cpp

```
#include <Arduino.h>
// ----- PIN DEFINITIONS -----
const int PIN_CURRENT_SENSOR = 34; // ACS712 analog output (ADC1 input, input-only pin)
const int PIN_RELAY           = 26; // Relay control (Active LOW)
const int PIN_BUTTON          = 27; // Emergency stop / start-stop button (INPUT_PULLUP)

const int PIN_LED_GREEN       = 16; // Motor running
const int PIN_LED_YELLOW      = 17; // Motor stopped
const int PIN_LED_BLUE        = 18; // Low current
const int PIN_LED_RED         = 19; // High current (fault)

// ----- RELAY / LOGIC LEVELS -----
const int RELAY_ON = LOW; // Active LOW relay module
const int RELAY_OFF = HIGH;

// ----- ADC / SENSOR CONSTANTS -----
const float ADC_VREF = 3.3f; // ESP32 ADC reference voltage
const int ADC_RESOLUTION = 4095; // 12-bit ADC (0-4095)
const float VOLTAGE_DIVIDER_RATIO = 1.0f; // Set > 1.0 if using a divider (see note above)
const float ACS712_SENSITIVITY = 0.066f; // V per A, ACS712-30A
const float NOISE_FLOOR_CURRENT = 0.03f; // Ignore current readings below this (A)

const int CALIBRATION_SAMPLES = 800; // Number of samples used at startup (500-1000 range)
const int CURRENT_SAMPLES = 200; // Samples averaged per readCurrent() call (raised for a noisy ACS712 signal)

const float CURRENT_EMA_ALPHA = 0.2f;
```

```

const unsigned long OVERCURRENT_CONFIRM_MS = 150;
const unsigned long STARTUP_GRACE_MS = 500;

// ----- PROTECTION THRESHOLDS -----
const float HIGH_CURRENT_THRESHOLD = .50f; // Amps - trips protection immediately
const float LOW_CURRENT_THRESHOLD = 0.05f; // Amps - below this, Blue LED indicates low current

// ----- TIMING CONSTANTS -----
const unsigned long DEBOUNCE_DELAY_MS = 50; // Button software debounce
const unsigned long AUTO_RESTART_DELAY_MS = 3000; // Wait time after fault clears before restart
const unsigned long PRINT_INTERVAL_MS = 200; // Serial status print interval

// ----- SYSTEM STATE MACHINE -----
enum SystemState {
    STATE_STOPPED, // Motor off, idle (default / after emergency stop released back to idle)
    STATE_RUNNING, // Motor on, current within normal range
    STATE_LOW_CURRENT, // Motor on, but current below LOW_CURRENT_THRESHOLD
    STATE_HIGH_CURRENT, // Fault: overcurrent detected, motor forcibly disabled
    STATE_EMERGENCY_STOP // Motor stopped by user via the push button
};

SystemState systemState = STATE_STOPPED;

// ----- GLOBAL RUNTIME VARIABLES -----
float zeroVoltage = 0.0f; // Calibrated sensor offset voltage (no-load output)
float lastCurrent = 0.0f; // Most recent filtered current reading (A)
float lastVoltage = 0.0f; // Most recent raw sensor voltage (V)

bool motorRunning = false; // True when relay is energized / motor commanded ON
unsigned long motorStartTime = 0; // millis() timestamp of the last startMotor() call

// Button debounce state
int lastRawButtonReading = HIGH;
int stableButtonState = HIGH;
unsigned long lastDebounceTime = 0;

// High current trip / auto-restart timing
unsigned long highCurrentClearedTime = 0; // millis() timestamp when current became safe again
bool waitingForRestart = false;

// Overcurrent confirmation timing (prevents false trips from ADC noise)
unsigned long overCurrentStartTime = 0;
bool overCurrentTimerRunning = false;

// Exponential moving average state for the current reading
float smoothedCurrent = 0.0f;
bool smoothedCurrentInitialized = false;

// Serial print timing
unsigned long lastPrintTime = 0;

// ----- FUNCTION PROTOTYPES -----
// Required because PlatformIO (main.cpp) does not auto-generate these

void setupPins();
void calibrateSensor();
void readCurrent();
void startMotor();
void stopMotor();
void handleButton();

```

```

void onButtonPressed();
void protectionLogic();
void updateLEDs();
void printStatus();
const char* stateToString(SystemState state);

void setup() {
  Serial.begin(115200);
  delay(200);

  setupPins();
  stopMotor();           // Ensure motor/relay start in a known OFF state
  calibrateSensor();     // Establish zero-current reference voltage

  systemState = STATE_STOPPED;
  updateLEDs();

  Serial.println(F("====="));
  Serial.println(F(" ESP32 DC Motor Protection System - Initialized"));
  Serial.println(F("====="));
}

void loop() {
  handleButton();        // Check for user start/stop requests
  readCurrent();         // Update lastCurrent / lastVoltage
  protectionLogic();      // Apply high/low current protection rules
  updateLEDs();          // Reflect current state on the LEDs
  printStatus();         // Periodic serial reporting
}

void setupPins() {
  pinMode(PIN_CURRENT_SENSOR, INPUT);
  pinMode(PIN_RELAY, OUTPUT);
  pinMode(PIN_BUTTON, INPUT_PULLUP);

  pinMode(PIN_LED_GREEN, OUTPUT);
  pinMode(PIN_LED_YELLOW, OUTPUT);
  pinMode(PIN_LED_BLUE, OUTPUT);
  pinMode(PIN_LED_RED, OUTPUT);

  digitalWrite(PIN_RELAY, RELAY_OFF);
  digitalWrite(PIN_LED_GREEN, LOW);
  digitalWrite(PIN_LED_YELLOW, LOW);
  digitalWrite(PIN_LED_BLUE, LOW);
  digitalWrite(PIN_LED_RED, LOW);
}

void calibrateSensor() {
  Serial.println(F("Calibrating current sensor... keep motor OFF."));

  double total = 0.0;

  for (int i = 0; i < CALIBRATION_SAMPLES; i++) {
    total += analogRead(PIN_CURRENT_SENSOR);
    delayMicroseconds(500); // Small spacing between samples for ADC stability
  }

  float averageRaw = total / CALIBRATION_SAMPLES;
  zeroVoltage = (averageRaw / ADC_RESOLUTION) * ADC_VREF * VOLTAGE_DIVIDER_RATIO;

  Serial.print(F("Calibration complete. Zero-current voltage = "));
  Serial.print(zeroVoltage, 4);
  Serial.println(F(" V"));
}

```

```

void readCurrent() {
    long total = 0;

    for (int i = 0; i < CURRENT_SAMPLES; i++) {
        total += analogRead(PIN_CURRENT_SENSOR);
    }

    float averageRaw = (float)total / CURRENT_SAMPLES;
    float sensorVoltage = (averageRaw / ADC_RESOLUTION) * ADC_VREF * VOLTAGE_DIVIDER_RATIO;

    float current = fabs(sensorVoltage - zeroVoltage) / ACS712_SENSITIVITY;

    // Ignore very small values to reduce noise from ADC jitter
    if (current < NOISE_FLOOR_CURRENT) {
        current = 0.0f;
    }

    if (!smoothedCurrentInitialized) {
        smoothedCurrent = current;
        smoothedCurrentInitialized = true;
    } else {
        smoothedCurrent = (CURRENT_EMA_ALPHA * current) + ((1.0f - CURRENT_EMA_ALPHA) *
smoothedCurrent);
    }

    lastVoltage = sensorVoltage;
    lastCurrent = smoothedCurrent;
}

void startMotor() {
    digitalWrite(PIN_RELAY, RELAY_ON);
    motorRunning = true;
    motorStartTime = millis();
}

void stopMotor() {
    digitalWrite(PIN_RELAY, RELAY_OFF);
    motorRunning = false;
}

void handleButton() {
    int rawReading = digitalRead(PIN_BUTTON);

    // Reset debounce timer whenever the raw reading changes
    if (rawReading != lastRawButtonReading) {
        lastDebounceTime = millis();
    }

    // If the reading has been stable longer than the debounce delay,
    // accept it as the new stable state
    if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY_MS) {
        if (rawReading != stableButtonState) {
            stableButtonState = rawReading;

            // Falling edge = button just pressed (pulled up, pressed = LOW)
            if (stableButtonState == LOW) {
                onButtonPressed();
            }
        }
    }

    lastRawButtonReading = rawReading;
}

```

```

}

// Handles a single confirmed button press event
void onButtonPressed() {
    bool motorCurrentlyActive =
        (systemState == STATE_RUNNING || systemState == STATE_LOW_CURRENT);

    if (motorCurrentlyActive) {
        stopMotor();
        systemState = STATE_EMERGENCY_STOP;
        waitingForRestart = false; // Cancel any pending auto-restart
        Serial.println(F(">>> Emergency Stop button pressed: motor stopped."));
    } else if (systemState == STATE_STOPPED || systemState == STATE_EMERGENCY_STOP) {
        startMotor();
        systemState = STATE_RUNNING;
        Serial.println(F(">>> Start button pressed: motor running."));
    }
}

void protectionLogic() {
    bool withinStartupGrace = motorRunning && (millis() - motorStartTime <
STARTUP_GRACE_MS);

    // ---- HIGH CURRENT FAULT (confirmed over time, not a single sample) ----
    if (!withinStartupGrace && motorRunning && lastCurrent > HIGH_CURRENT_THRESHOLD) {
        if (!overCurrentTimerRunning) {
            overCurrentTimerRunning = true;
            overCurrentStartTime = millis();
        } else if (millis() - overCurrentStartTime >= OVERCURRENT_CONFIRM_MS) {
            stopMotor();
            systemState = STATE_HIGH_CURRENT;
            waitingForRestart = false;
            overCurrentTimerRunning = false;
            Serial.print(F("!!! WARNING: High current detected ("));
            Serial.print(lastCurrent, 3);
            Serial.println(F(" A). Motor disabled for protection."));
        }
        return;
    } else {
        overCurrentTimerRunning = false;
    }

    // ---- MANAGE AUTO-RESTART AFTER A TRIP ----
    if (systemState == STATE_HIGH_CURRENT) {
        if (lastCurrent <= HIGH_CURRENT_THRESHOLD) {
            if (!waitingForRestart) {
                waitingForRestart = true;
                highCurrentClearedTime = millis();
                Serial.println(F("Current back to safe levels. Waiting to auto-restart..."));
            } else if (millis() - highCurrentClearedTime >= AUTO_RESTART_DELAY_MS) {
                startMotor();
                systemState = STATE_RUNNING;
                waitingForRestart = false;
                Serial.println(F(">>> Auto-restart complete. Motor running."));
            }
        } else {
            // Current spiked again while waiting; reset the restart timer
            waitingForRestart = false;
        }
    }
    return; // Do not evaluate low-current logic while tripped
}

```

```

// ---- LOW CURRENT DETECTION (motor keeps running) ----
if (motorRunning) {
    if (lastCurrent < LOW_CURRENT_THRESHOLD) {
        systemState = STATE_LOW_CURRENT;
    } else {
        systemState = STATE_RUNNING;
    }
}
}

void updateLEDs() {
    bool motorIsOn = (systemState == STATE_RUNNING || systemState == STATE_LOW_CURRENT);

    digitalWrite(PIN_LED_GREEN, motorIsOn ? HIGH : LOW);
    digitalWrite(PIN_LED_YELLOW, motorIsOn ? LOW : HIGH);

    digitalWrite(PIN_LED_BLUE, (systemState == STATE_LOW_CURRENT) ? HIGH : LOW);
    digitalWrite(PIN_LED_RED, (systemState == STATE_HIGH_CURRENT) ? HIGH : LOW);
}

void printStatus() {
    unsigned long now = millis();
    if (now - lastPrintTime < PRINT_INTERVAL_MS) {
        return;
    }
    lastPrintTime = now;

    Serial.print(F("Current: "));
    Serial.print(lastCurrent, 3);
    Serial.print(F(" A | Voltage: "));
    Serial.print(lastVoltage, 3);
    Serial.print(F(" V | Motor: "));
    Serial.print(motorRunning ? F("RUNNING") : F("STOPPED"));
    Serial.print(F(" | Relay: "));
    Serial.print(motorRunning ? F("ON") : F("OFF"));
    Serial.print(F(" | Protection: "));
    Serial.println(stateToString(systemState));
}

const char* stateToString(SystemState state) {
    switch (state) {
        case STATE_STOPPED:      return "STOPPED";
        case STATE_RUNNING:      return "NORMAL";
        case STATE_LOW_CURRENT:   return "LOW_CURRENT";
        case STATE_HIGH_CURRENT:  return "HIGH_CURRENT_TRIP";
        case STATE_EMERGENCY_STOP: return "EMERGENCY_STOP";
        default:                  return "UNKNOWN";
    }
}

}

```

Design notes: the fault is latched in software so a brief overcurrent spike cannot silently clear itself and re-energize the motor into the same fault — the operator must consciously press the reset button once the underlying issue is fixed. All pin numbers and thresholds are defined once, at the top of the file, as named constants, so the code can be adapted to a different motor, relay, or pin layout by changing a single line rather than hunting through the logic.

Testing & Verification (Definition of Done)

Serial Monitor Results:

```
Current: 0.277 A | Voltage: 2.416 V | Motor: RUNNING | Relay: ON | Protection: NORMAL
Current: 0.278 A | Voltage: 2.419 V | Motor: RUNNING | Relay: ON | Protection: NORMAL
Current: 0.290 A | Voltage: 2.416 V | Motor: RUNNING | Relay: ON | Protection: NORMAL
Current: 0.294 A | Voltage: 2.418 V | Motor: RUNNING | Relay: ON | Protection: NORMAL
Current: 0.281 A | Voltage: 2.417 V | Motor: RUNNING | Relay: ON | Protection: NORMAL
Current: 0.302 A | Voltage: 2.419 V | Motor: RUNNING | Relay: ON | Protection: NORMAL
Current: 0.290 A | Voltage: 2.417 V | Motor: RUNNING | Relay: ON | Protection: NORMAL
Current: 0.278 A | Voltage: 2.417 V | Motor: RUNNING | Relay: ON | Protection: NORMAL
Current: 0.280 A | Voltage: 2.417 V | Motor: RUNNING | Relay: ON | Protection: NORMAL
```

```
Current: 0.240 A | Voltage: 2.383 V | Motor: STOPPED | Relay: OFF | Protection: HIGH_CURRENT_TRIP
Current: 0.240 A | Voltage: 2.382 V | Motor: STOPPED | Relay: OFF | Protection: HIGH_CURRENT_TRIP
Current: 0.248 A | Voltage: 2.382 V | Motor: STOPPED | Relay: OFF | Protection: HIGH_CURRENT_TRIP
Current: 0.238 A | Voltage: 2.385 V | Motor: STOPPED | Relay: OFF | Protection: HIGH_CURRENT_TRIP
Current: 0.239 A | Voltage: 2.381 V | Motor: STOPPED | Relay: OFF | Protection: HIGH_CURRENT_TRIP
Current: 0.231 A | Voltage: 2.384 V | Motor: STOPPED | Relay: OFF | Protection: HIGH_CURRENT_TRIP
Current: 0.240 A | Voltage: 2.382 V | Motor: STOPPED | Relay: OFF | Protection: HIGH_CURRENT_TRIP
Current: 0.234 A | Voltage: 2.387 V | Motor: STOPPED | Relay: OFF | Protection: HIGH_CURRENT_TRIP
Current: 0.235 A | Voltage: 2.382 V | Motor: STOPPED | Relay: OFF | Protection: HIGH_CURRENT_TRIP
```

Definition of Done Item	Verification Procedure	Result
E-stop actually stops the motor instantly	Press the E-stop button while the motor is running under normal load; confirm the motor stops within the same instant (hardware cutoff, not just a software flag) and the Red LED lights.	Pass / Fail
Overcurrent protection triggers and cuts the relay when current gets too high	Load or briefly slow the motor shaft to push current above the High threshold (1.5 A in this design); confirm the relay opens and the Yellow LED latches on until manually reset.	Pass / Fail
LEDs are wired up correctly with proper resistors	Verify each of the four LEDs lights only in its correct state, and measure that each resistor limits current to a safe value for a standard 5 mm LED ($\approx 10\text{--}15\text{ mA}$ at 3.3 V through a 330 Ω resistor).	Pass / Fail
Code is clean and easy to modify	Confirm all pins and thresholds are named constants at the top of main.cpp, functions are single-purpose, and no debug/test code remains in the final version.	Pass / Fail
A clear, readable wiring schematic drawn in any software	Produce the schematic in Fritzing, EasyEDA, or KiCad (see Section 10) showing every connection listed in Section 6, with clear labels and a legend.	Pending — attach schematic file

Recommendations & Schematic Notes

For the final wiring schematic required by the Definition of Done, any of the following free tools produce a submission-ready diagram:

- **Fritzing:** the fastest option for a beginner-friendly breadboard-style view that closely matches the physical build; ideal if the schematic just needs to be clear and readable rather than manufacturing-grade.
- **EasyEDA (online, free):** produces a proper electrical schematic with standard component symbols and can export directly to PDF/PNG; a good middle ground between speed and professionalism.
- **KiCad:** a full professional EDA tool; more setup effort but the right choice if the schematic may later be turned into an actual PCB.

Additional recommendations for a stronger, safer build:

- **Keep the E-stop hardware-wired:** as described in Section 6.3, rather than relying on the firmware alone to detect and act on it — a frozen or crashed microcontroller must never be able to keep the motor running.
- **Latch the overcurrent fault:** rather than letting the system silently retry, so a repeating fault cannot cycle the relay on and off (which is hard on both the relay contacts and the motor).
- **Add flyback protection:** on the relay coil (most pre-built relay modules already include this) to protect the driver transistor from voltage spikes when the coil de-energizes.
- **Keep the motor's power path and the ESP32's logic power path clearly separated:** on the breadboard, with a single shared ground point, to avoid noise on the current-sensor reading.

Conclusion

This design meets every requirement of the Project 2 brief: the motor is powered through a supply sized with an explicit margin, its current is monitored continuously and compared to a chosen, documented threshold, an overcurrent condition opens a relay to remove power, a hardware-wired E-stop button provides an instantaneous and firmware-independent safety cutoff, and four LEDs give unambiguous, at-a-glance status feedback. The firmware is deliberately small, uses named constants for every tunable value, and latches faults so the system fails safe rather than cycling. Combined with a schematic produced in Fritzing, EasyEDA, or KiCad, this report and its accompanying code satisfy the project's full Definition of Done.

References

- **Texas Instruments:** INA219 Zero-Drift, Bidirectional Current/Power Monitor datasheet.
- **Adafruit:** Adafruit INA219 Current Sensor Breakout — guide and Arduino library documentation.
- **PlatformIO:** PlatformIO Documentation — Espressif 32 platform and VS Code IDE integration, platformio.org
- **Espressif Systems:** ESP32 Series Datasheet — GPIO, ADC, and I2C peripheral reference, espressif.com
- **NEMA / IEC motor-protection practice:** General industry guidance on sizing thermal-magnetic motor overload protection relative to full-load current, used here to justify the 125–150% trip-point rule of thumb.